Computing for Analytical Work

Session 2 — What is running?

Block 3 — Python interpreters, packages, imports, and environments

Knowledge check 2 — 10 minutes, closed book, 3 questions

- Covers Session 1 and Homework 1: paths, working directory, terminal, `git status`
- Laptops closed. Paper only.
- Three short questions. Diagnostic, not tricky.
- Worth 5% of the final grade
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer goes. Then we begin.

This block in one sentence

"Python" means a particular interpreter with a particular set of installed packages.

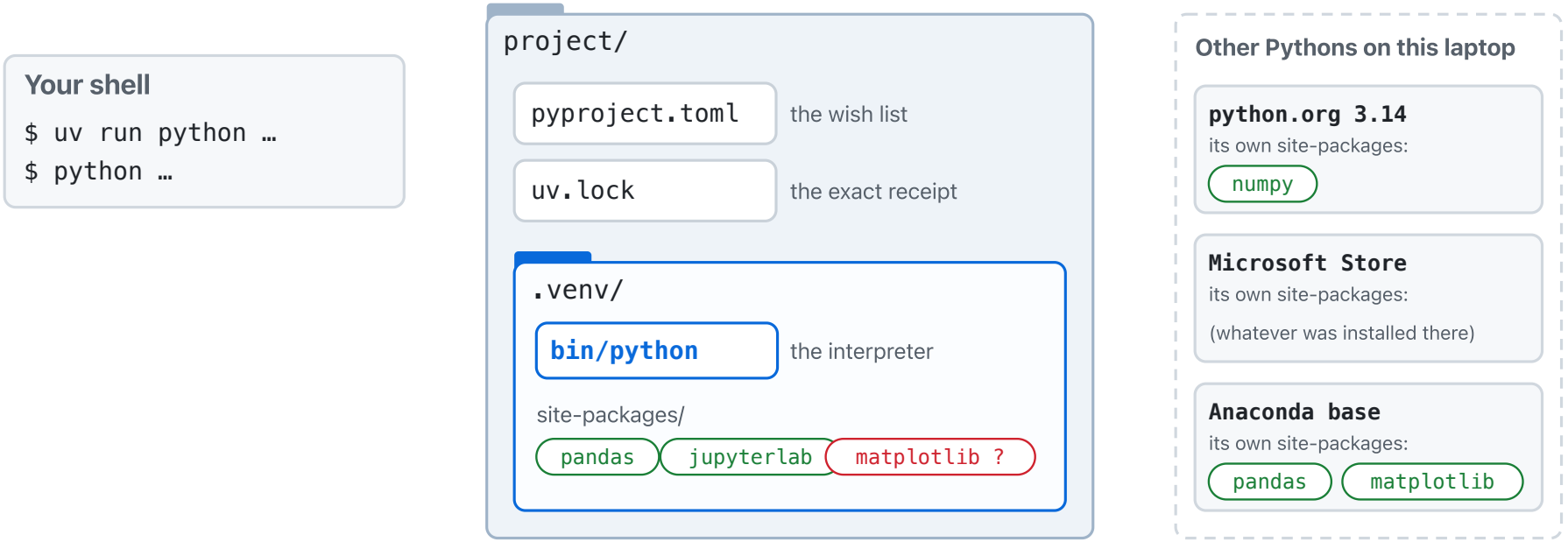
Not a language. Not an icon. A specific program, on a specific path, with a specific list of packages it can see.

What we cover

- The reveal: what `uv run` has been doing all week
- Interpreter, script, notebook
- Packages, `import`, and `sys.path`
- Installing into a specific environment; why installs "fail"
- Many Pythons on one machine, and how to check which is active
- `uv`: the one workflow this program uses
- Git thread: `git add`, `git commit`
- AI thread, stretch, and Lab 3

You have been typing `uv run` for a week

`uv run python scripts/summarize.py` — in the setup check, Lab 1, Lab 2, Homework 1. Every README said: *"You will learn what `uv run` does in Session 2. For now, type it exactly."* It is Session 2. This map is the answer.



Interpreter, script, notebook

- **Interpreter** — a program, one executable file, that reads Python code and runs it
- **Script** — a `.py` file the interpreter reads top to bottom, once, then exits
- **Notebook** — a long-running interpreter (the *kernel*) you feed one cell at a time

```
uv run python -c "import sys; print(sys.executable)"
```

That prints the path of the interpreter that ran the line. It is a file. You can `ls` it.

What a package is

- A folder of Python code somebody else wrote, plus metadata
- `pandas`, `matplotlib`, `jupyter` — each is a folder on disk, installed *somewhere*
- Installed means: copied into a folder **that one interpreter knows to look in**

```
uv run python -c "import pandas; print(pandas.__file__)"
```

That prints where pandas lives. Note the path. It will matter in five slides.

What `import` actually does

```
import pandas
```

1. The interpreter takes a list of folders called `sys.path`
2. It walks that list, in order, looking for a folder or file named `pandas`
3. First match wins. No match: `ModuleNotFoundError`

`import` is a **search**. It is not a download, and it is not a question about your laptop.

`sys.path` belongs to *that* interpreter

`import pandas` is a search through one interpreter's sys.path, in order. First match wins.

sys.path of **`.venv/bin/python`**

1. `project/`
2. `.venv/lib/python3.13/site-packages` — `pandas ✓`

found: stop, import it

sys.path of **`/usr/bin/python3`**

1. `/usr/lib/python3.9/`
2. `/usr/lib/python3.9/site-packages/`
3. `/usr/lib/python3/dist-packages/` — `no pandas`

end of the list: ModuleNotFoundError

Same code. Two interpreters. Each searched its own list and told the truth.

So "is pandas installed?" is an incomplete question. **Installed for which interpreter?**

Installing into a specific environment

- An **environment** is one interpreter plus the folders it searches. That is all it is.
- Installing a package puts it into **one** environment
- This course: every project has its own environment, in `.venv/`, inside the project folder

```
uv add matplotlib
```

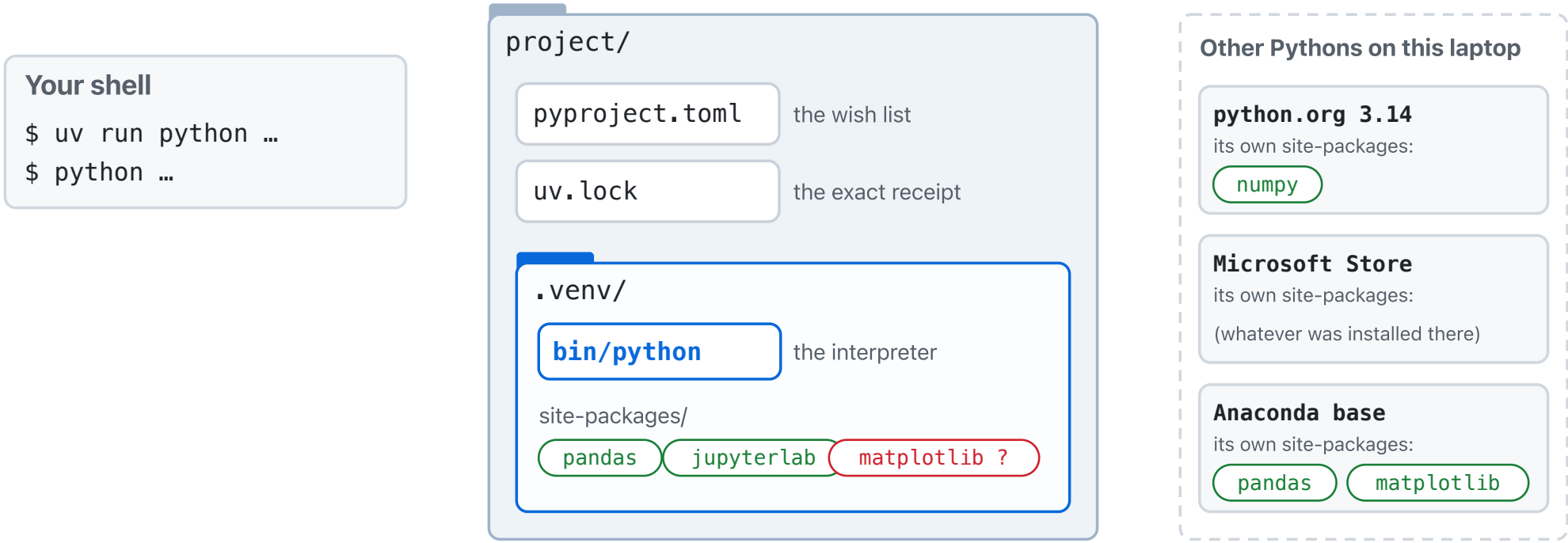
That installs matplotlib into *this project's* environment, and records that it did.

Why installs "fail"

```
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, pillow, packaging, numpy, kiwisolver, f
onttools, cycler, python-dateutil, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.64.0 kiwisolver-1.5.1
 matplotlib-3.11.1 numpy-2.5.3 packaging-26.3 pillow-12.3.0 pyparsing-3.3.2 python-dat
eutil-2.9.0.post0 six-1.17.0
(other) anna@laptop project $ uv run python -c "import matplotlib"
warning: `VIRTUAL_ENV=/Users/Shared/anna/other` does not match the project environment
 path `.venv` and will be ignored; use `--active` to target the active environment ins
tead
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
(other) anna@laptop project $
```

The install did not fail. It **succeeded, somewhere else.** A receipt from one Python, an error from another, one minute apart.

Many Pythons on one machine



Each has its own `sys.path`. Nothing coordinates them. **Which one runs is decided by which one you invoke.**

How the shell finds `python`: `PATH`

- `PATH` is a list of folders. The shell walks it in order; the first folder holding a `python` wins.
- `which python` prints that winner. Installing Anaconda, Homebrew, or the Store version puts a folder at the *front* of the list.
- `uv run` does not consult `PATH` for the interpreter. It goes to the project's `.venv/`.

Same idiom as `import`: **a list, walked in order, first match wins.**

How to check which Python is active

```
python --version              # which Python is on PATH right now
which python                  # where that Python actually lives
uv --version                  # confirm uv is installed
uv python list                # which Pythons uv knows about
uv run python -c "import sys; print(sys.executable)"   # the one inside the project env
```

Every one of these passes an argument to `python`. **Never type bare `python` in Git Bash**: on some setups it hangs with no error; on others it drops you into Python's `>>>` prompt. Neither is where you want to be.

Predict: two Pythons, side by side

```
python -c "import sys; print(sys.executable)"
uv run python -c "import sys; print(sys.executable)"
```

Same laptop, same folder, thirty seconds apart. **Will these two commands print the same path?**

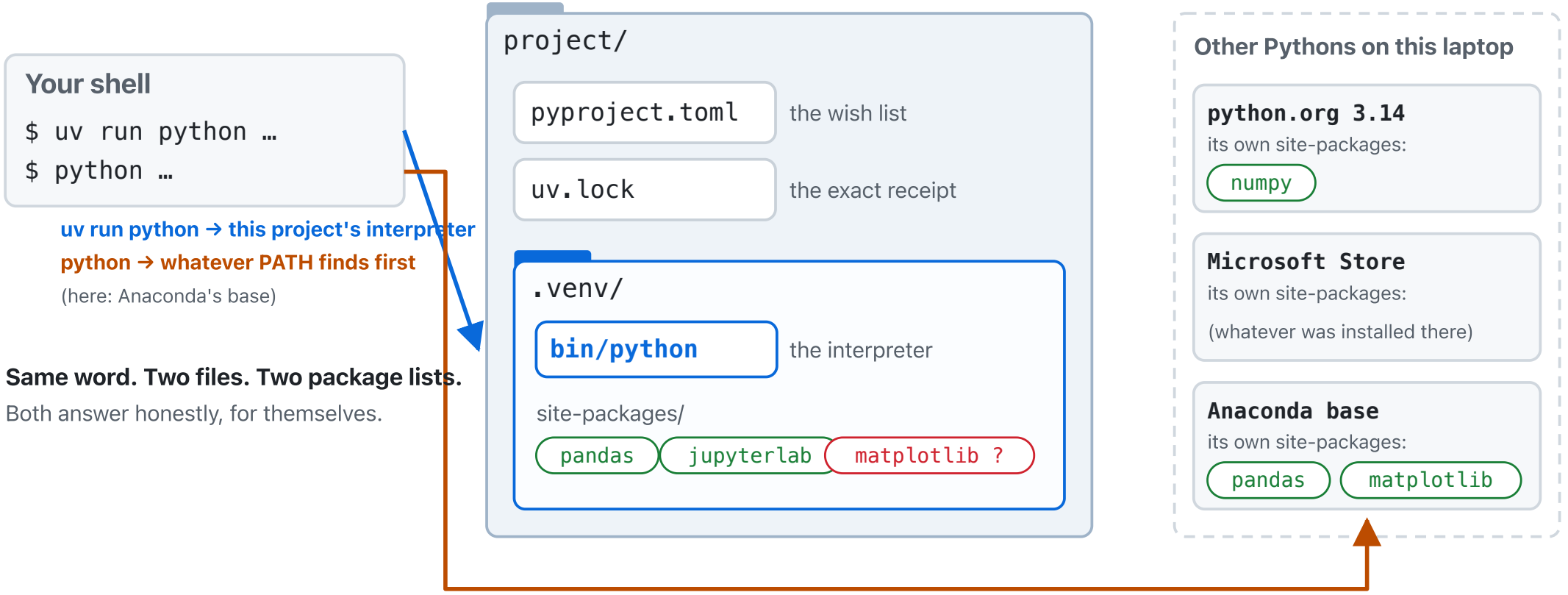
Write down *yes* or *no*, and one reason. Do not type yet.

Two Pythons — what happened

```
anna@laptop project $ python -c "import sys; print(sys.executable)"
/opt/homebrew/Caskroom/miniconda/base/bin/python
anna@laptop project $ uv run python -c "import sys; print(sys.executable)"
/Users/Shared/anna/project/.venv/bin/python3
anna@laptop project $ uv run python -c "import pandas; print(pandas.__file__)"
/Users/Shared/anna/project/.venv/lib/python3.13/site-packages/pandas/__init__.py
anna@laptop project $ uv run python -c "import matplotlib"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
anna@laptop project $
```

Two paths. The first is whatever the shell found; the second is *this project's* interpreter. Then the same import, asked of each.

What you just saw, on the map



Part 2 — `uv`: the one workflow

The four commands

```
uv sync                          # install everything the project declares
uv add pandas                    # declare and install a new dependency
uv run python script.py          # run a script inside the project's environment
uv run jupyter lab               # launch JupyterLab inside the project's environment
```

Always from inside the project folder. That is the whole workflow for this program.

`pyproject.toml` and `uv.lock`, in plain words

pyproject.toml

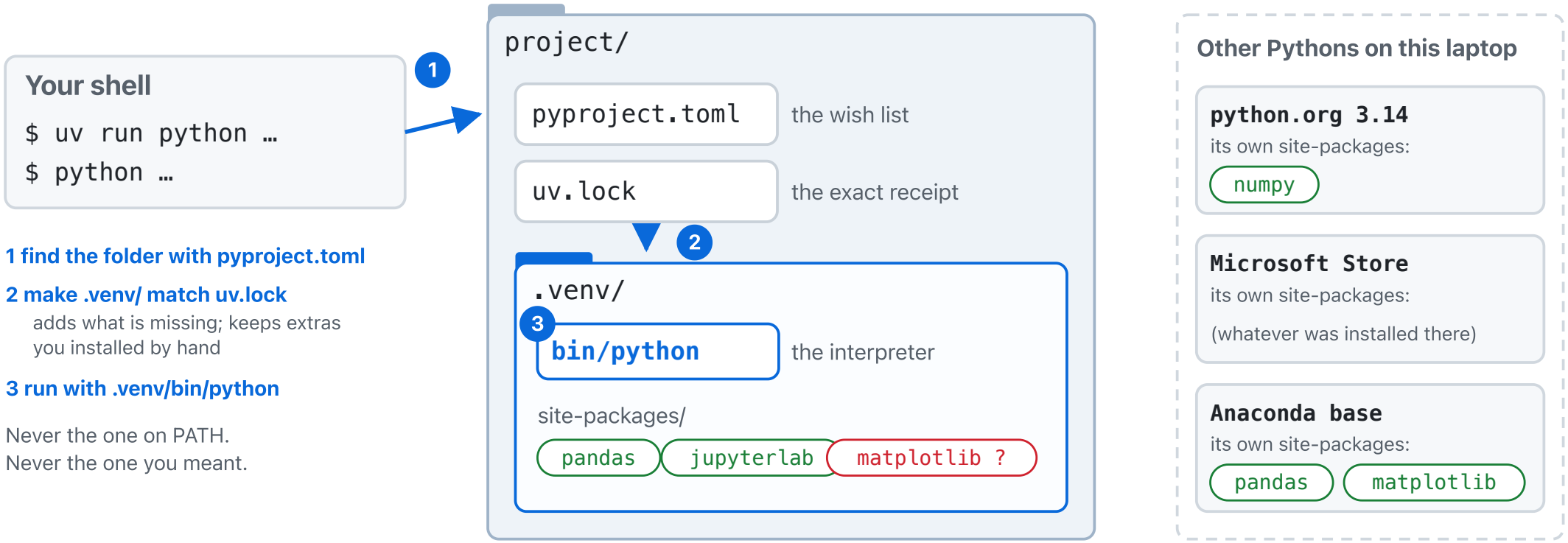
```
[project]
name = "ecbs5293-lab03-environments"
version = "0.1.0"
description = "ECBS5293 lab 3: it imports on …
requires-python = ">=3.10"
dependencies = [
    "pandas",
    "matplotlib",
]
```

uv.lock (one of its entries)

```
[[package]]
name = "matplotlib"
version = "3.10.9"
source = { registry = "https://pypi.org/simple" }
dependencies = [
sdist = { url = "https://files.pythonhosted.org/pa …
```

1. The wish list: names, no versions. You edit this with uv add.
2. The receipt: the exact version that satisfied the wish, for every package, including ones you never named.
3. A hash of the bytes: uv sync installs the same thing on every machine, and refuses anything else.

So what was `uv run` doing?



1 find the folder with pyproject.toml

2 make .venv/ match uv.lock

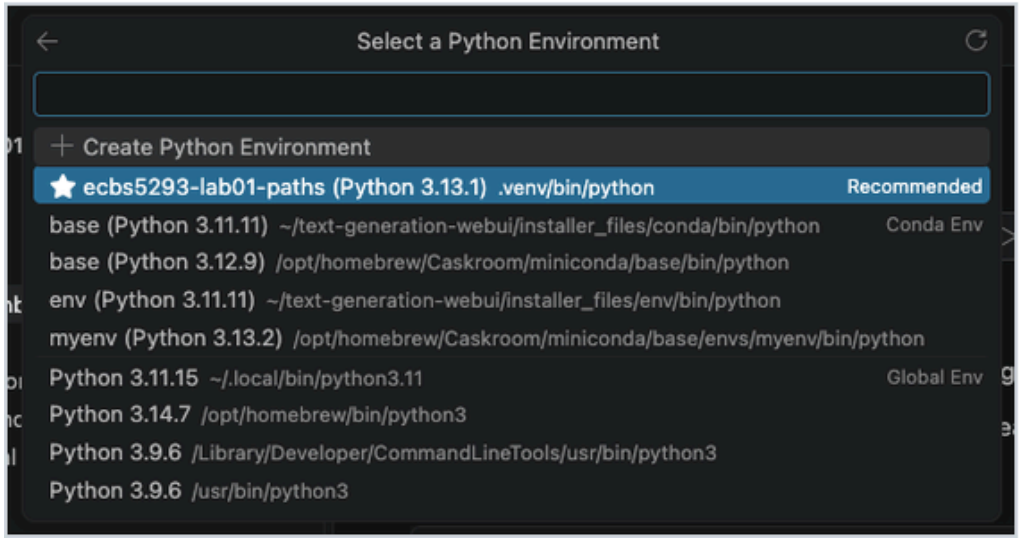
adds what is missing; keeps extras you installed by hand

3 run with .venv/bin/python

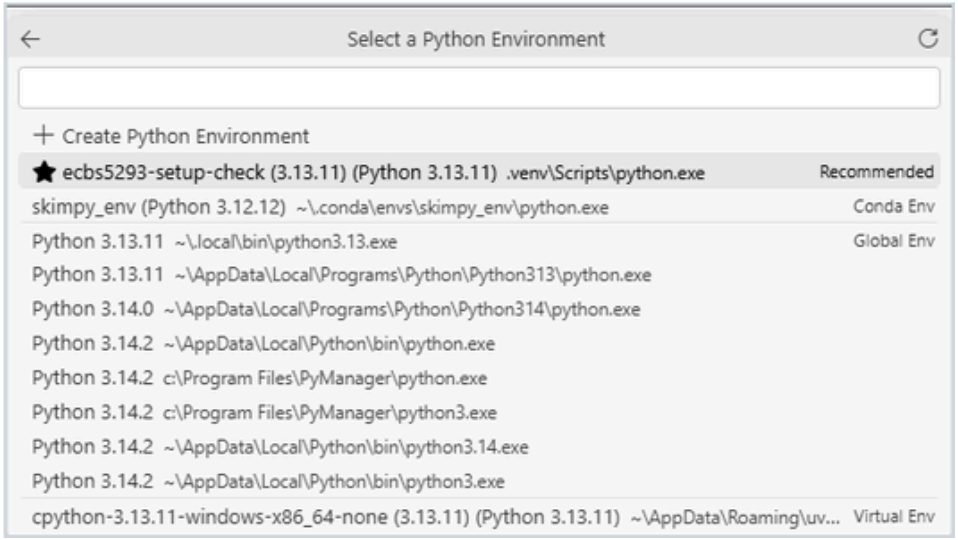
Never the one on PATH.
Never the one you meant.

The notebook standard: the project's own kernel

macOS: .venv/bin/python



Windows: .venv\Scripts\python.exe



1. Open the **project folder** (never the lone `.ipynb`). 2. Kernel picker → the interpreter inside **`.venv/`**.
3. Verify: `import sys; sys.executable` in a cell prints a path inside *this project's* `.venv/`.

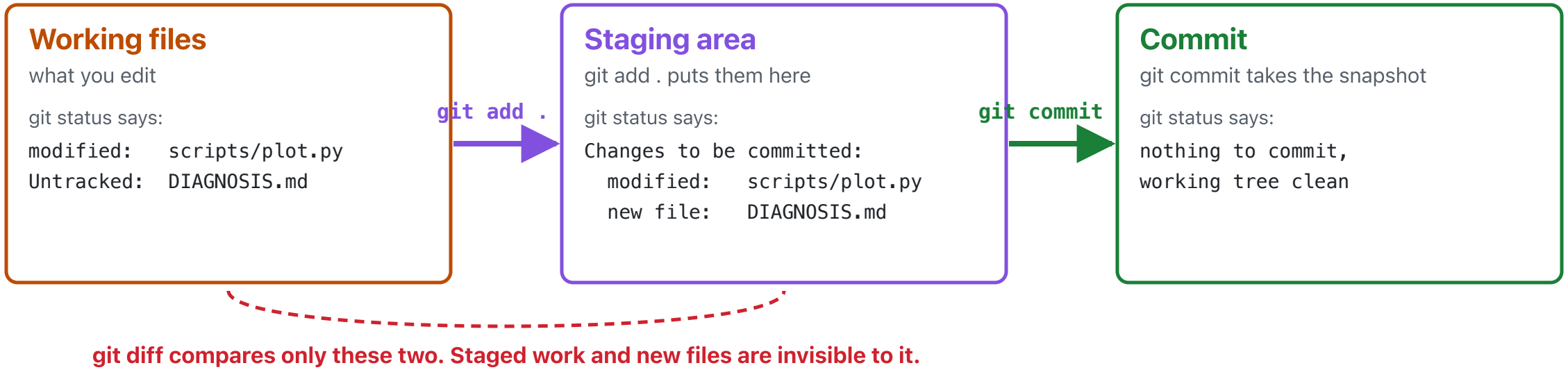
Alternative: `uv run jupyter lab` from the project folder offers the project's kernel by default. Verify it the same way.

In the wild: `requirements.txt`

- Older repos, blog posts, and other courses ship `requirements.txt` instead of `pyproject.toml`
- Same job, older file: *make this project's packages available to its interpreter*
- `uv` handles it: `uv pip install -r requirements.txt`
- **Not lectured.** The Lab 3 stretch task and the reference in the README cover it.

If a README says `pip install -r requirements.txt` and there is no such file, that README is wrong.

A whiff of Git — save a checkpoint




```
git status           # what changed?
git add .            # stage it: "include these in the next snapshot"
git commit -m "Declare matplotlib in pyproject; fix README"
```

A commit is a **checkpoint after a working fix**. Commit when it works. Not before, not "at the end".

After you commit: `git status` first, then `git diff`

```
anna@laptop project $ git status --short
 M pyproject.toml
?? DIAGNOSIS.md
anna@laptop project $ git add .
anna@laptop project $ git diff
anna@laptop project $ git status --short
A  DIAGNOSIS.md
M  pyproject.toml
anna@laptop project $ git commit -q -m "Declare matplotlib; add diagnosis note"
anna@laptop project $ git status
On branch main
nothing to commit, working tree clean
anna@laptop project $
```

- `git status` covers everything: unstaged edits, staged work, and files Git has never seen. "nothing to commit, working tree clean" is the completeness check.
- `git diff` alone shows **unstaged edits to tracked files**. Staged work and new files are invisible to it.

AI thread — ask for evidence, not for a fix

Good questions for an assistant today:

- *"What evidence distinguishes 'package not installed' from 'installed in a different environment'?"*
- *"What does* `uv sync` *actually do? What is* `pyproject.toml` *? How is* `uv` *different from* `pip` *or* `conda` *?"*
- *"Here is* `sys.executable` *from my terminal and from my notebook. They differ. What does that mean?"*

Then run the inspection yourself and paste the real output.

The active interpreter decides what is true

- AI can suggest causes. It cannot see your machine.
- `sys.executable` shows which interpreter answered; printing `sys.path` shows where it searched.
- If the assistant says "it's installed" and the interpreter says `ModuleNotFoundError`, the interpreter is right.

Sources of truth today: the filesystem, the interpreter, `uv`'s output. Everything else is a hypothesis.

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five.

Lab 3 — It imports on my machine, but not here

First 15 minutes: no AI. Chat windows and assistant panels closed; VS Code itself stays open. Terminal, filesystem, and `uv` output only. Search engines are fine.

Why: every failure today is one an assistant diagnoses instantly. The reflex we want is *"check which interpreter,"* not *"ask what to check."* The exam has no chat window. Solve it in five minutes and go to checkoff; the fifteen minutes are for looking, not for suffering.

TAs will ask you to close a window. That is not a penalty.

The symptom, and how to run it

```
anna@laptop ecbs5293-lab03-environments $ uv run python scripts/plot.py
Traceback (most recent call last):
  File "/Users/Shared/anna/ecbs5293-lab03-environments/scripts/plot.py", line 8, in <m
odule>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
anna@laptop ecbs5293-lab03-environments $
```

```
git clone <your Lab 3 repo> && cd <the folder> && uv sync && uv run python scripts/plot.py
```

The README says `pip install -r requirements.txt`. There is no such file. Your job: which interpreter is looking, what the project *declares*, and why they disagree.

What you produce, and the checkoff

1. `uv run python scripts/plot.py` runs clean, from a fresh `uv sync`
2. `pyproject.toml` declares the missing dependency (`uv add`, not a hand edit)
3. README setup instructions are correct for *this* repo
4. One commit of the working state; `git status` reports a clean tree afterward
5. `DIAGNOSIS.md`: symptom, cause, evidence (paste real output), change, verification
6. Checkoff: explain it to a TA in about a minute. No notes. Then one what-if question about environments.

Stretch: fresh clone + `uv sync`; then the `requirements.txt` reference in the README. Reset instructions are in the README; both resets destroy work.